



对外提供

Orbbec OpenNI2.0-SDK Android 接口使用规范

[针对 Android 平台]

密级

机密

修订历史

版本	修订者	日期	备注
V1.0	李小斌	2018/06/22	首发版本
V2.0	柳洞兵	2019/05/26	修订版本
V3.0	李小斌	2019/07/03	文档更新： 1. 文档整合优化 2. 增加内容： 13.3 硬件 D2C 调用规范; 13.4 网络授权规范;
V3.1	李小斌	2020/01/09	1.接口调用规范; 2.新增扩展接口;



目录

修订历史	1
1 前言	5
2 Orbbec-SDK	5
2.1 SDK 框架介绍	5
2.2 库文件	5
2.3 ini 文件	6
2.4 SDK 接入	6
3 SDK 初始化	6
3.1 初始化方式	6
3.2 初始化接口	7
4 请求打开 UsbDevice	7
4.1 请求打开 UsbDevice 接口	7
4.2 具体实现方式	8
5 Device 打开	9
5.1 Device 枚举	9
5.2 Device 匹配	9
5.3 Device 打开	10
5.4 Device 打开业务实现	10
6 VideoStream 创建	10
6.1 VideoStream 创建	11
6.2 VideoMode 获取	11
6.3 VideoMode 设置	11
6.4 VideoStream 创建业务实现	12
6.5 VideoStream 创建流类型实现	12
6.5.1 Depth 流业务实现	12
6.5.2 IR 流业务实现	13
6.5.3 Color 流业务实现	13
7 VideoStream 开流	13
7.1 主动获取方式	14
7.2 回调获取方式	15
8 VideoStream 停流	15
9 VideoStream 销毁	16
10 Device 关闭	16
11 UsbDevice 关闭	17
12 SDK 释放	17
13 接口协同调用规范	18
13.1 主动获取数据流程规范	18
13.1.1 主动获取数据流程图	18
13.1.2 主动获取数据流规范	19
13.2 回调流程规范	20
13.2.1 回调模式获取图像流程图	20



13.2.2	回调流程规范	21
13.3	硬件 D2C 调用规范	22
13.4	网络授权规范	23
14	SDK 释放	23
15	纯净 IR 图获取	24
16	扩展 API 接口	24
16.1	Device 相关基础接口	24
16.1.1	获取设备信息	24
16.1.2	设置帧同步	24
16.1.3	获取帧同步状态	25
16.1.4	设置激光状态	25
16.1.5	获取激光使能状态	25
16.1.6	设置 LDP 状态	25
16.1.7	设置 IR 泛光灯状态	25
16.1.8	获取 IR 泛光灯状态	25
16.1.9	设置 IR 增益	26
16.1.10	获取 IR 增益	26
16.1.11	获取 USB 类型	26
16.1.12	DeviceUSBSpeed	26
16.1.13	获取相机保存标定参数	26
16.1.14	获取固件版本号	26
16.1.15	获取设备 SN 号	27
16.1.16	硬件 D2C 支持判断	27
16.1.17	设置硬件 D2C 模式	27
16.1.18	获取硬件 D2C 模式	27
16.1.19	设置 Depth 和 IR 交替输出模式	27
16.1.20	获取 Depth 和 IR 交替输出模式	28
16.2	VideoStream 相关基础接口	28
16.2.1	设置镜像	28
16.2.2	获取镜像状态	28
16.2.3	设置 VideoMode	28
16.2.4	获取 VideoMode	29
16.2.5	设置 Depth 最大值	29
16.2.6	设置 Depth 最小值	29
16.2.7	设置数据输入格式	29
16.2.8	设置 HoleFilter	30
16.2.9	设置 SoftFilter	30
17	配置文件介绍	30
17.1	Openni.ini 配置文件	30
17.2	Orbbec.ini 配置文件	31
18	Libuvc 应用	34
18.1	数据监听器注册	34
18.2	数据回调	34
19	Demo 参考	34



20	结束	35
----	----------	----

ORBEC Confidential

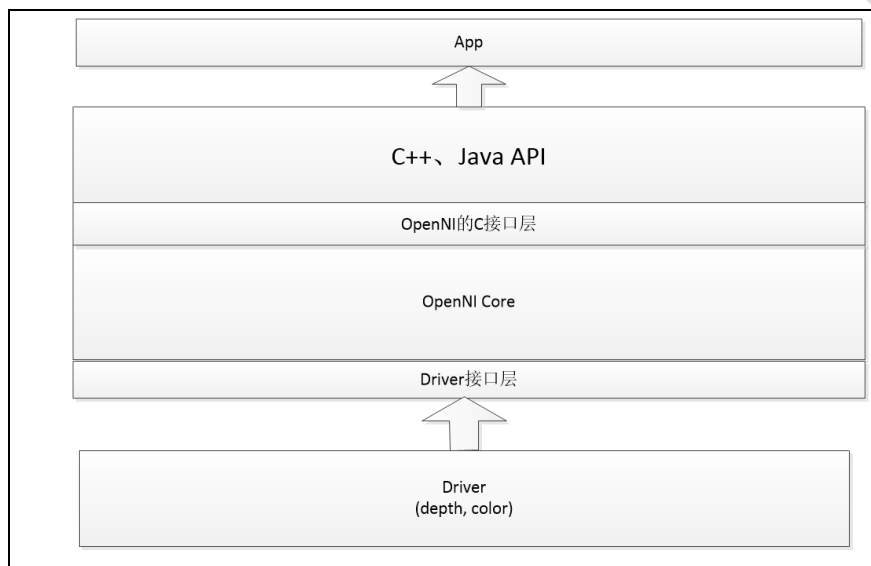
1 前言

为了使用户正确的、高效的在自己的项目中快速接入 Orbbec Android 平台的 OpenNI2.0-SDK(简称 Orbbec-SDK)，防止在使用 Orbbec-SDK 相关 API 接口的过程中由于不规范的调用而引起其他问题，故拟定次文档来规范关键性 API 的调用。此文档只针对 Android 平台。

2 Orbbec-SDK

2.1 SDK 框架介绍

OpenNI 2 分层设计，OpenNI 2 框架如下：



Openni2.0 Driver 层：

- 平台适配：隔离 windows 和 linux 系统之间的差异，包括线程，内存管理，事件/互斥锁，定时器等；
- 适配 windows 和 linux 下 usb 通信接口：linux 使用 libusb 通信，windows 使用 orbbec 自己的驱动 obdrv.sys 对上层提供统一的接口；
- Usb 通信：包括设备枚举，usb 控制命令实现，视频流的接收，解析。

Openni 层： 主要封装对外的获取视频流接口及参数设置的接口。

2.2 库文件

Orbbec-SDK 提供给用户的包括两部分内容，5 个 so 动态库文件和 1 个 jar 文件：



对外提供



2.3 ini 文件

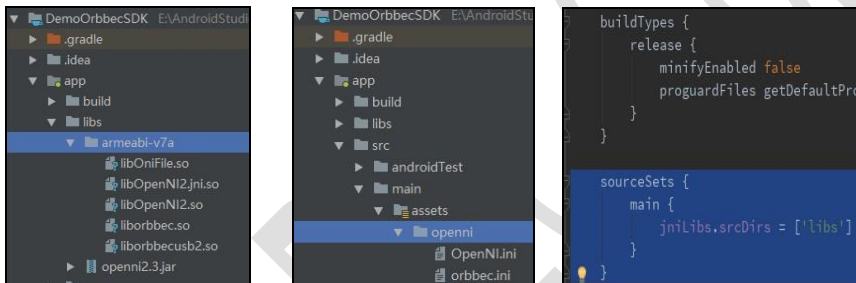
两个 ini 配置文件:



2.4 SDK 接入

将库文件放在接入工程 Module 的 libs 目录下, ini 文件放在接入工程的 Module 的 app->src->main->asset->openni 目录下, 在 Module 的 build.gradle 中配置重新指定 jniLibs 目录, 最后同步整个工程。

如图所示:

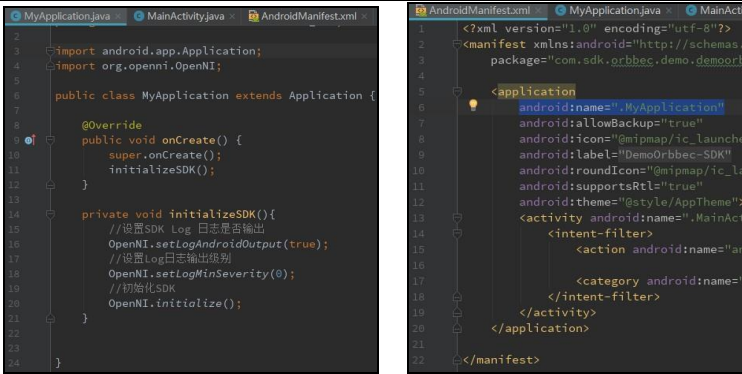


3 SDK 初始化

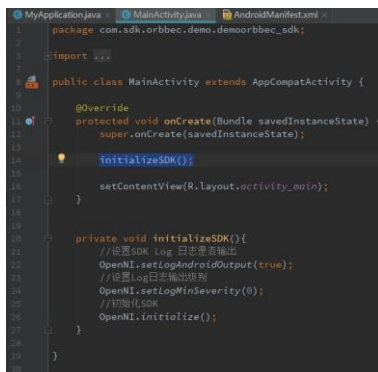
3.1 初始化方式

在调用 Orbbec-SDK 之前必须要初始化, 一般在程序运行开始时进行, 比如 Android 应用的 onCreate 方法中。初始化方法如下:

1. 方法 1: 自定义 Application 初始化, 自定义的 Application 需要在 Module 的 AndroidManifest.xml 文件中注册。



2. 方法 2: Activity 的 onCreate 中完成。



3.2 初始化接口

初始化调用接口:

接口类	API	参数	描述
OpenNI	OpenNI.setLogAndroidOutput(true);	true:输出 Android 日志 false: 反之	设置 SDK Log 日志是否输出
	OpenNI.setLogMinSeverity(0);	0 - Verbose; 1 - Info; 2 - Warning; 3 - Error;	设置 Log 日志输出级别
	OpenNI.Initialize();		初始化 SDK, 必须调用

4 请求打开 UsbDevice

请求打开 UsbDevice 是 Android 系统授权的过程, 在 Android 系统中操作 UsbDevice 必须要先申请权限。

4.1 请求打开 UsbDevice 接口

➤ 在 Orbec-SDK 中, 请求 UsbDevice 过程:



1. OpenNIHelper 枚举 Orbbec 的 usb 设备，如果未找到 Orbbec 的 usb 设备，回调未找到设备，程序终止；
2. 如果“1”找到 Orbbec usb 设备，OpenNIHelper 向系统发送申请 UsbDevice 权限的广播；
3. OpenNIHelper 注册广播监听器监听系统授权后 UsbDevice 请求状态返回；
4. OpenNIHelper 调用用户注册的监听器返回 UsbDevice 请求结果，请求结果包括 UsbDevice 打开成功、打开失败和设备未发现三种结果。

➤ 请求 UsbDevice 接口：

接口类	API	参数	描述
OpenNIHelper	OpenNIHelper(Context context);	Context 建议使用： getApplicationContext()	OpenNIHelper 初始化
	requestDeviceOpen(OpenNIHelper.DeviceOpenListener listener)	DeviceOpenListener	请求并打开 UsbDevice 的连接

➤ UsbDevice 请求结果回调接口：

接口类	API	参数	描述
DeviceOpenListener	void onDeviceOpened(UsbDevice device);	UsbDevice device	返回请求成功的 UsbDevice
	void onDeviceOpenFailed(String msg);	String msg	失败返回错误信息回调此接口表示打开设备失败
	void onDeviceNotFound();	无	设备未找到回调

注意：如果 OpenNIHelper 没有枚举到设备，或者请求权限失败，通过 DeviceOpenListener 回调 UsbDevice 请求结果。

4.2 具体实现方式

1. OpenNIHelper 初始化，实现方式：

OpenNIHelper openNIHelper=new OpenNIHelper(getApplicationContext());

可以根据自己的需求在程序初始化时调用，且只需要调用一次即可，例如在 onCreate 完成最后初始化。

2. 请求 UsbDevice 并注册状态监听：

实现方式：

openNIHelper.requestDeviceOpen(deviceOpenListener);

注册回调获取设备监听器，如图：


```
public class MainActivity extends AppCompatActivity implements View.OnClickListener{  
    private OpenNIHelper openNIHelper;  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        initializeSDK();  
        setContentView(R.layout.activity_main1);  
        initView();  
  
        openNIHelper = new OpenNIHelper(getApplicationContext());  
        openNIHelper.requestDeviceOpen(deviceOpenListener);  
    }  
  
    OpenNIHelper.DeviceOpenListener deviceOpenListener = new OpenNIHelper.DeviceOpenListener() {  
        @Override  
        public void onDeviceOpened(UsbDevice usbDevice) {  
        }  
  
        @Override  
        public void onDeviceOpenFailed(String s) {  
        }  
  
        @Override  
        public void onDeviceNotFound() {  
        }  
    }  
};
```

3. 如果是 Pro 系列深度设备（Color 是标准的 UVC 摄像头），需要自己申请 color 摄像头的权限(Android6.0 以上就需要动态申请权限)，并重写 Activity 的 onRequestPermissionsResult 函数，Android6.0 以上需要用户授权的情况下才可以获取访问 Camera 设备，因此 Pro 系列深度设备的 UVC 摄像头需要在 onRequestPermissionsResult 回调中打开。

5 Device 打开

这里的 Device 是即将打开使用的 Orbbec 设备，要使用正确的 Device 需要经过枚举、匹配、打开三个步骤。

5.1 Device 枚举

Orbbec Device 枚举是通过 OpenNI.enumerateDevices()接口实现，实现方式：

```
List<DeviceInfo> deviceInfos=OpenNI.enumerateDevices();
```

具体调用接口：

接口类	API	参数	描述
OpenNI	enumerateDevices ();	无	返回 List<DeviceInfo> 设备列表信息

5.2 Device 匹配

“4.2” 获取到已经申请权限的 UsbDevice 后，与通过 OpenNI 枚举(“5.1”)到 Orbbec 的 DeviceInfo 进行 VID, PID 对比匹配，查找出 Orbbec 的 Usb Device。

匹配逻辑：

详细匹配

```
(deviceInfos.getUsbProductId() == device.getProductId() && deviceInfo.getUsbVendorId() == device.getVendorId())
```

5.3 Device 打开

“5.2” 查找出 Orbbec 的 Device 后打开。

实现方式：

```
Device device = Device.open();
```

具体调用接口：

接口类	API	参数	描述
Device	open ();	无	返回打开成功的 Device

5.4 Device 打开业务实现

```
private void requestDevice(){
    Log.i(TAG, "msg: "requestDeviceOpen");
    openNIHelper.requestDeviceOpen(deviceOpenListener);
}

OpenNIHelper.DeviceOpenListener deviceOpenListener = new OpenNIHelper.DeviceOpenListener() {

    @Override
    public void onDeviceOpened(UsbDevice usbDevice) {
        List<DeviceInfo> deviceInfos=OpenNI.enumerateDevices();
        if(deviceInfos.size()==0){
            return;
        }

        for (int i = 0; i < deviceInfos.size(); i++) {
            if(deviceInfos.get(i).getUsbProductId()==usbDevice.getProductId()){
                device = Device.open();
                break;
            }
        }

    }

    @Override
    public void onDeviceOpenFailed(String s) {

    }

    @Override
    public void onDeviceNotFound() {
        Toast.makeText(context: SyncMainActivity.this, text: "Orbbec-Device not found!", Toast.LENGTH_SHORT).show();
    }

};
```

6 VideoStream 创建

Device 打开后，创建 VideoStream， VideoStream 的创建分为 Depth、Color(走 OpenNI2.0 协议的)、IR 三种类型。

6.1 VideoStream 创建

实现方式：

```
VideoStream stream = VideoStream.create(device, SensorType.DEPTH);
```

这里的 Device 是步骤“5”得到的，具体调用接口。

接口类	API	参数	描述
VideStream	create(device, SensorType.DEPTH);	Device	“5.3”成功打开的 Device
		SensorType	VideoStream 类型
SensorType	IR(1); COLOR(2); DEPTH(3);	无	根据 SensorType 创建不同类型的 VideoStream 类型，包括 Depth、IR 和 Color(by OpenNI)。

6.2 VideoMode 获取

VideoStream 创建成功，可以获取、设置对应创建的 Stream 的分辨率、数据输出格式、帧率等。

实现方式：`List<VideoMode> videoModes = stream.getSensorInfo().getSupportedVideoModes();`

此处的 stream 是“6.1”成功创建的 VideoStream。

具体调用接口：

接口类	API	参数	描述
List<VideoMode>	getSensorInfo().getSupportedVideoModes()	无	返回当前创建 VideStream 支持的 VideoMode 类型列表

6.3 VideoMode 设置

实现方式：

```
stream.setVideoMode(videoMode);
```

此处的 stream 是“6.1”得到的 VideoStream，videoMode 是“6.2”得到的。

具体接口调用：

接口类	API	参数	描述
VideoStream	setVideoMode(videoMode)	VideoMode	设置 VideStream 输出的数据流模式

注意：

Orbbec.ini 中可以配置默认的数据流的分辨率，输出格式,如果代码中有设置 VideoMode，则会覆盖该配置。

6.4 VideoStream 创建业务实现

VideoStream 整体实现过程如图：

```
OpenNIHelper.DeviceOpenListener deviceOpenListener = new OpenNIHelper.DeviceOpenListener() {  
    @Override  
    public void onDeviceOpened(UsbDevice usbDevice) {  
        List<DeviceInfo> deviceInfos=OpenNI.enumerateDevices();  
        if(deviceInfos.size()==0){  
            return;  
        }  
        for (int i = 0; i < deviceInfos.size() ; i++) {  
            if(deviceInfos.get(i).getUsbProductId()==usbDevice.getProductId()){  
                device = Device.open();  
                break;  
            }  
        }  
        if(device!=null){  
            Log.i(TAG, msg: "createStream");  
            videoStream=VideoStream.create(device, SensorType.DEPTH);  
            List<VideoMode> videoModes = videoStream.getSensorInfo().getSupportedVideoModes();  
            for (int j = 0; j < videoModes.size(); j++) {  
                VideoMode videoMode = videoModes.get(j);  
                if(videoMode.getResolutionX()== 400 && videoMode.getResolutionY()== 640  
                    && videoMode.getPixelFormat() == PixelFormat.DEPTH_1_MM){  
                    videoStream.setVideoMode(videoMode);  
                    break;  
                }  
            }  
        }  
    }  
    @Override  
    public void onDeviceOpenFailed(String s) {  
    }  
    @Override  
    public void onDeviceNotFound() {  
        Toast.makeText( context: SyncMain3Activity.this, text: "Orbbec-Device not found!", Toast.LENGTH_S  
    }  
}
```

6.5 VideoStream 创建流类型实现

6.5.1 Depth 流业务实现

业务实现描述

1. 创建 Depth 流类型；
2. 获取 Depth 流支持的 VideoMode 列表；
3. 设置 Depth 流输出的 VideoMode 类型，VideoMode 类型通过类型过滤检索后设置。

如：D2 VideoMode 设置查找分辨率 w:400，h:640，PixelFormat：DEPTH_1_MM。

```
private void createStream(){  
    if(device!=null){  
        Log.i(TAG, msg: "createStream");  
        videoStream=VideoStream.create(device, SensorType.DEPTH);  
        List<VideoMode> videoModes = videoStream.getSensorInfo().getSupportedVideoModes();  
        for (int j = 0; j < videoModes.size(); j++) {  
            VideoMode videoMode = videoModes.get(j);  
            if(videoMode.getResolutionX()== GlobalDef.RES_DEPTH_WIDTH && videoMode.getResolutionY()==GlobalDef.RES_DEPTH_HEIGHT  
                && videoMode.getPixelFormat() == PixelFormat.DEPTH_1_MM){  
                videoStream.setVideoMode(videoMode);  
                break;  
            }  
        }  
    }  
}
```

6.5.2 IR 流业务实现

业务实现描述

4. 创建 IR 流类型；
 5. 获取 IR 流支持的 VideoMode 列表；
 6. 设置 IR 流输出的 VideoMode 类型，VideoMode 类型通过类型过滤检索后设置。
- 如：D2 VideoMode 设置查找分辨率 w:640, h:400, PixelFormat: GRAY16。

```
private void createStream(){
    if(device!=null){
        Log.i(TAG, msg: "createStream");
        videoStream=VideoStream.create(device, SensorType.IR);
        List<VideoMode> videoModes = videoStream.getSensorInfo().getSupportedVideoModes();
        for (int j = 0; j < videoModes.size(); j++) {
            VideoMode videoMode = videoModes.get(j);
            if(videoMode.getResolutionX()== GlobalDef.RES_DEPTH_WIDTH && videoMode.getResolutionY()==GlobalDef.RES_DEPTH_HEIGHT
                && videoMode.getPixelFormat() == PixelFormat.GRAY16){
                videoStream.setVideoMode(videoMode);
                break;
            }
        }
    }
}
```

6.5.3 Color 流业务实现

业务实现描述

7. 创建 Color 流类型；
 8. 获取 Color 流支持的 VideoMode 列表；
 9. 设置 Color 流输出的 VideoMode 类型，VideoMode 类型通过类型过滤检索后设置。
- 如：P3X VideoMode 设置查找分辨率 w:384, h:640, PixelFormat: RGB888 或 YUYV。

注意事项： P3X 该 RGB 依赖主路 UVC RGB，主路 UVC RGB 输出后该 RGB 才可输出。

```
private void createStream(){
    if(device!=null){
        Log.i(TAG, msg: "createStream");
        videoStream=VideoStream.create(device, SensorType.COLOR);
        List<VideoMode> videoModes = videoStream.getSensorInfo().getSupportedVideoModes();
        for (int j = 0; j < videoModes.size(); j++) {
            VideoMode videoMode = videoModes.get(j);
            if(videoMode.getResolutionX()== GlobalDef.RES_DEPTH_WIDTH && videoMode.getResolutionY()==GlobalDef.RES_DEPTH_HEIGHT
                && videoMode.getPixelFormat() == PixelFormat.RGB888){
                videoStream.setVideoMode(videoMode);
                break;
            }
        }
    }
}
```

7 VideoStream 开流

VideoStream 数据流的获取(请求)分为主动获取和回调两种方式。

7.1 主动获取方式

实现方式(开启一个子线程去实现):

```
videoStream.start();
```

```
OpenNI.waitForAnyStream(streams,2000);( 2000 是超时等待的时间, 单位是 ms)
```

List<VideoStream>可以添加多个 VideoStream, 如下表格:

接口类	调用过程
VideoStream	<pre>VideoStream depthstream = VideoStream.create(device, SensorType.DEPTH); VideoStream colorstream = VideoStream.create(device, SensorType.COLOR); List<VideoStream> streams = new ArrayList<VideoStream>(); Streams.add(depthstream); Streams.add(colorstream);</pre>

具体接口调用:

接口类	API	参数	描述
OpenNI	waitForAnyStream(List<VideoStream> streams, int timeout);	streams: VideoStream 列表; timeout 等待一帧数据的超时时间;	这个函数是阻塞的, 直到一帧新的可用的数据返回
VideoStream	start();	无	开始流数据生成输出
VideoFrameRef	readFrame();	无	读取一帧数据流引用
	release();	无	释放一帧数据流引用
	readFrame()和 release() 接口必须一起使用, 读完后必须释放, 防止内存泄露。		

如图:

```
private void startStreamThread(){
    streamThread=new Thread(){
        @Override
        public void run() {
            List<VideoStream> streams = new ArrayList<>();
            streams.add(videoStream);
            videoStream.start();

            while (startStream){
                try {
                    OpenNI.waitForAnyStream(streams, timeout: 2000);
                } catch (TimeoutException e) {
                    e.printStackTrace();
                }

                synchronized (m_sync){
                    if(videoStream!=null){
                        VideoFrameRef videoFrameRef = videoStream.readFrame();
                        openNIView.update(videoFrameRef);
                        videoFrameRef.release();
                    }
                }
            }
        }
    };
    streamThread.start();
}
```

7.2 回调获取方式

通过设置回调接口实现(优先选择此方案)，实现方式：

```
videoStream.start();
videoStream.addNewFrameListener(newFrameListener);
videoFrameRef.readFrame();
videoFrameRef.release();
```

具体接口调用：

接口类	API	参数	描述
VideoStream	start();	无	打开流数据并输出
	addNewFrameListener(newFrameListener);	数据回调接口	设置数据流回调
VideoFrameRef	readFrame();	无	读取一帧数据流引用
	release();	无	释放一帧数据流引用

具体接口调用：

接口类	API	参数	描述
NewFrameListener	onFrameReady(VideoStream var1);	var1: 返回的流数据引用	返回可用的数据流引用，此回调函数中避免执行其他耗时操作。
VideoFrameRef 的 readFrame()和 release() 函数必须同时调用，防止内存泄露。			

详细接口调用如图：

```
VideoStream.NewFrameListener newFrameListener = new VideoStream.NewFrameListener() {
    @Override
    public void onFrameReady(VideoStream videoStream) {
        if(videoStream!=null){
            VideoFrameRef videoFrameRef = videoStream.readFrame();
            openNIView.update(videoFrameRef);
            videoFrameRef.release();
        }
    }
};
```

8 VideoStream 停流

VideoStream 停止流后如果要重新获取流数据，需调用开始流的接口，即：“[7](#)”步骤。

实现方式：

```
VideoStream.stop();
```

接口类	API	参数	描述
VideoStream	stop();	无	停止 VideoStream 流数据输出
	removeNewFrameListener(NewFrameListener streamListener)	NewFrameListener	取消数据异步回调接口监听

通过 [7.1](#) 主动方式实现，详细接口调用如图：

```
private void stopStream(){
    startStream = false;
    if (streamThread != null) {
        try {
            streamThread.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    if (videoStream != null) {
        videoStream.stop();
    }
}
```

通过 “[7.2](#)” 回调方式实现，需要移除回调代码：

VideoStream.removeNewFrameListener (newFrameListener);

详细接口调用如图：

```
private void stopStream(){
    if(videoStream!=null){
        videoStream.stop();
        videoStream.removeNewFrameListener(newFrameListener);
    }
}
```

9 VideoStream 销毁

实现方式：

VideoStream.destroy();

VideoStream 销毁后如果要再获取流需要重新创建 VideoStream，即：执行 “[6](#)” 步骤。

接口类	API	参数	描述
VideoStream	destroy();	无	销毁 VideoStream，与 VideoStream 创建对应，需同步调用。

详细接口调用如图：

```
private void destroyStream(){
    if(videoStream!=null){
        videoStream.destroy();
        videoStream=null;
    }
}
```

10 Device 关闭

实现方式：

device.close();

调用此接口后，如果要重新打开设备获取流数据，需从“4.2”步骤开始执行。

接口类	API	参数	描述
Device	close();	无	调用这个接口将关闭任何硬件设备

详细接口调用如图：

```
private void closeDevice(){
    if(device!=null){
        device.close();
    }
}
```

11 UsbDevice 关闭

实现方式：

openNIHelper.shutdown();

此 API 与步骤“4”对应，如果要重新获取设备需从步骤“4.1”开始执行。

接口类	API	参数	描述
OpenNIHelper	shutdown();	无	关闭 UsbDevice 的连接

详细接口调用如图：

```
private void shutdownOpenNIHelper(){
    if(openNIHelper!=null){
        Log.i(TAG, MSG: "shutdownOpenNIHelper");
        openNIHelper.shutdown();
        openNIHelper=null;
    }
}
```

12 SDK 释放

实现方式：

OpenNI.shutdown();

SDK 释放与文档“3”对应调用，在调用 OpenNI.shutdown()接口前必须释放所有的 ViewStream 和 Device 资源，即执行“8、9、10、11”步骤，若要重新使用 SDK 需从步骤“3”开始操作。若退出整个 APP 也许调用以上接口。

接口类	API	参数	描述
OpenNI	shutdown();	无	释放 Orbbec-SDK 资源

详细接口调用如图，建议调用处：

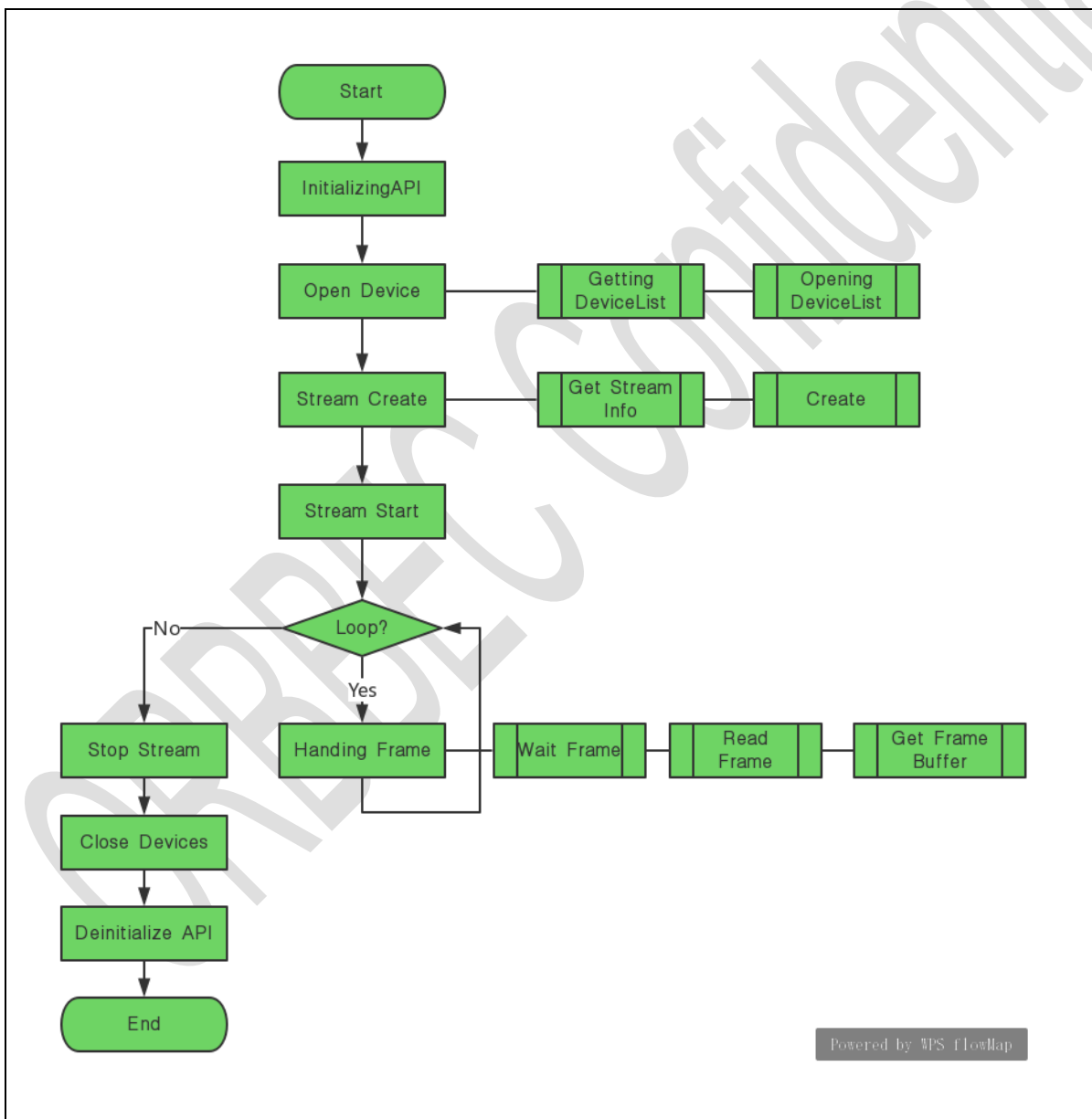
```
@Override
protected void onDestroy() {
    OpenNI.shutdown();
    super.onDestroy();
}
```

13 接口协同调用规范

接口协同调用规则指：规定“流数据”在成功获取和停止的原则下，规范正确合理的 API 调用顺序和调用一一对应原则。

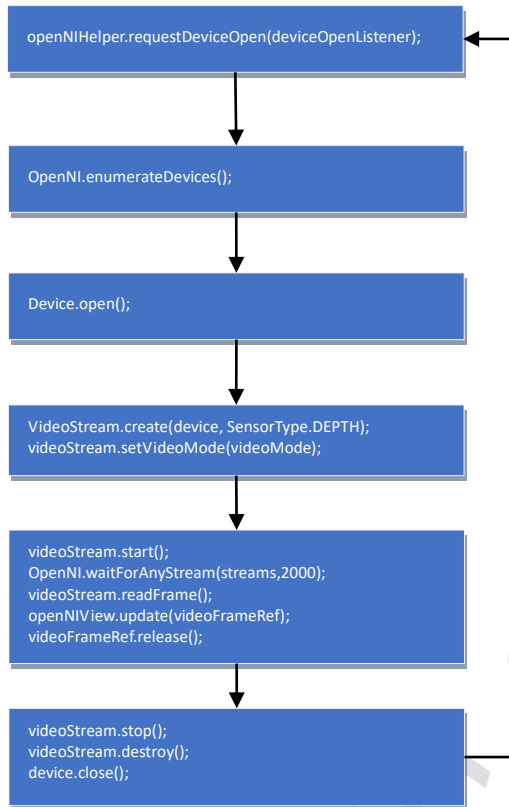
13.1 主动获取数据流程规范

13.1.1 主动获取数据流程图

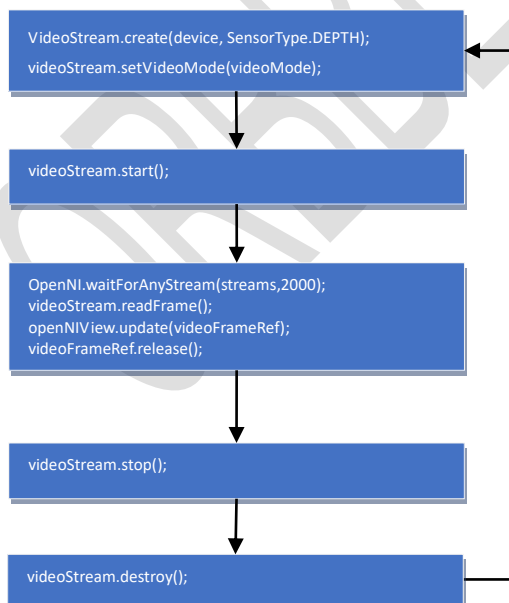


13.1.2 主动获取数据流规范

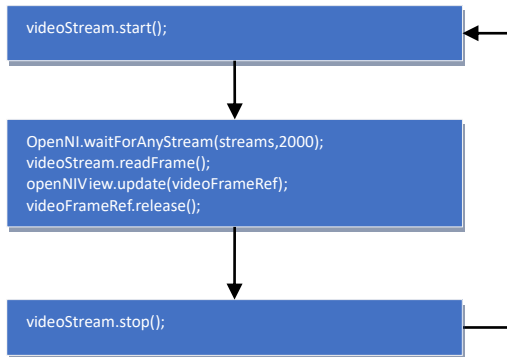
1. 规范 1，Device 关闭需重新请求获取并打开。



2. 规范 2，在 Device 不关闭的情况下，只需要执行 VideoStream 创建和销毁。

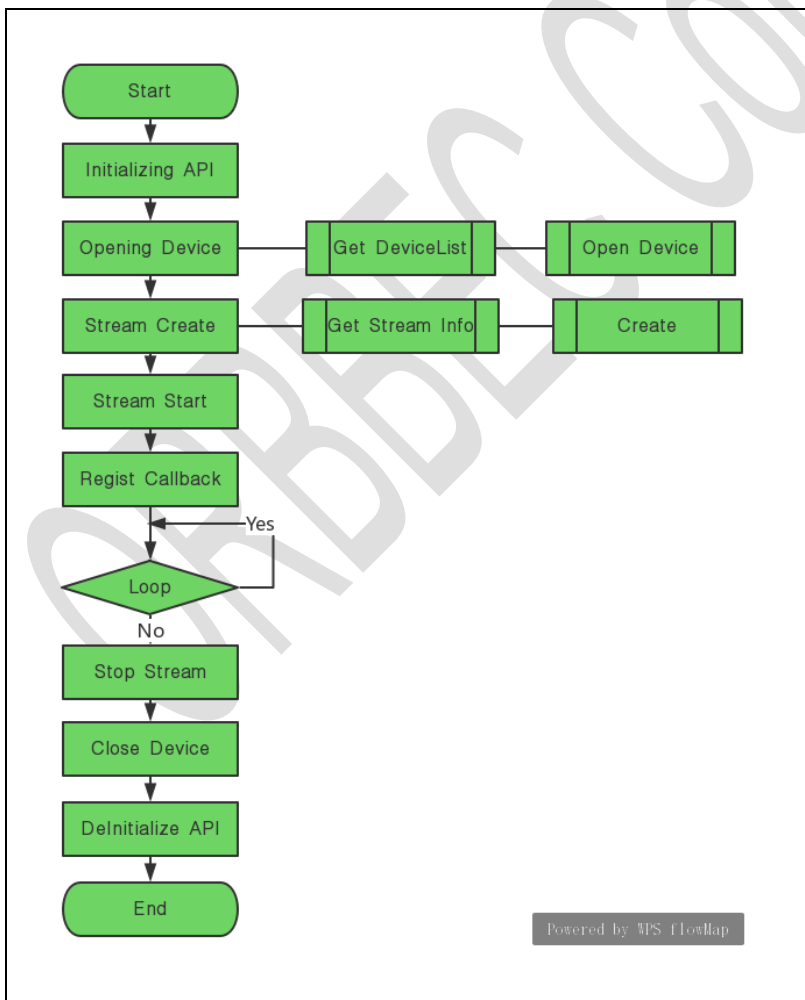


3. 规范 3, 在 **VideoStream** 没有销毁的情况下, 只需要执行 **VideoStream** 的 **start** 和 **stop** 函数即可, 最优选择方案:



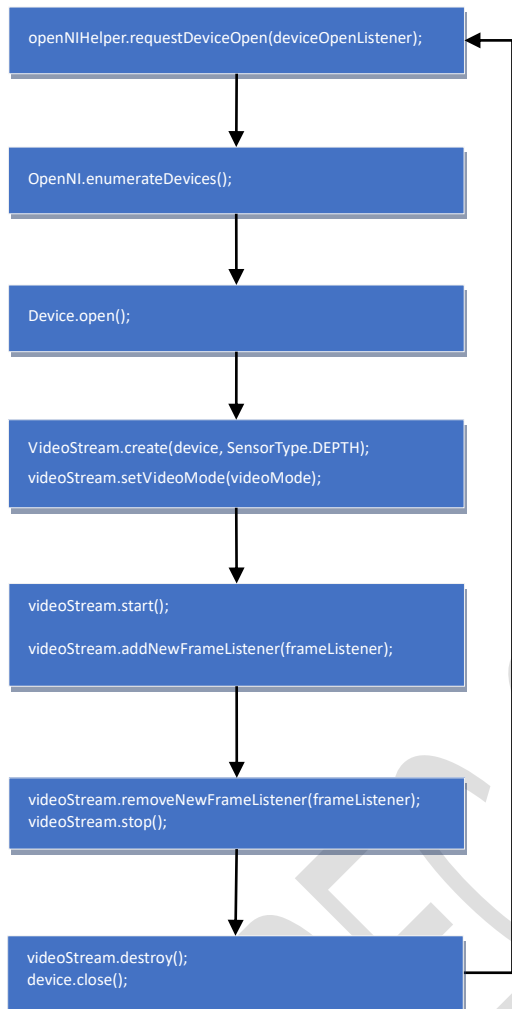
13.2 回调流程规范

13.2.1 回调模式获取图像流程图

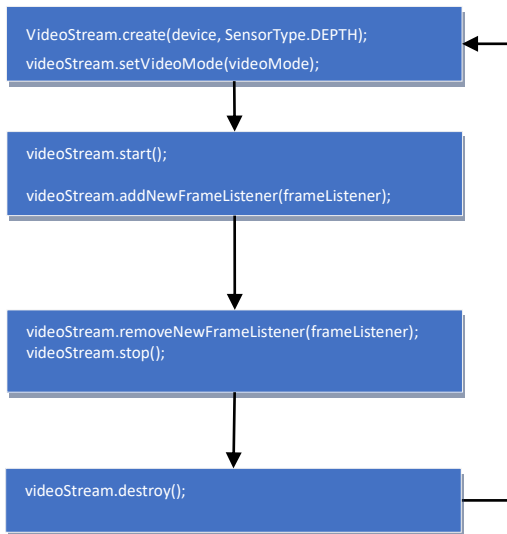


13.2.2 回调流程规范

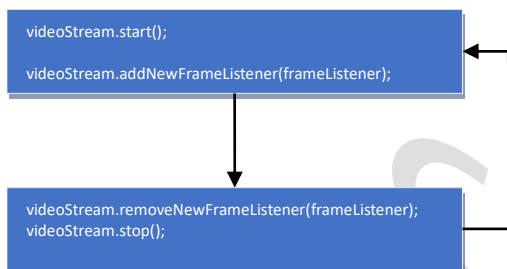
1. 规范 1，Device 关闭需重新获取 Device 并打开。



2. 规范 2，在 Device 不关闭的情况下，只需要执行创建和销毁 VideoStream。

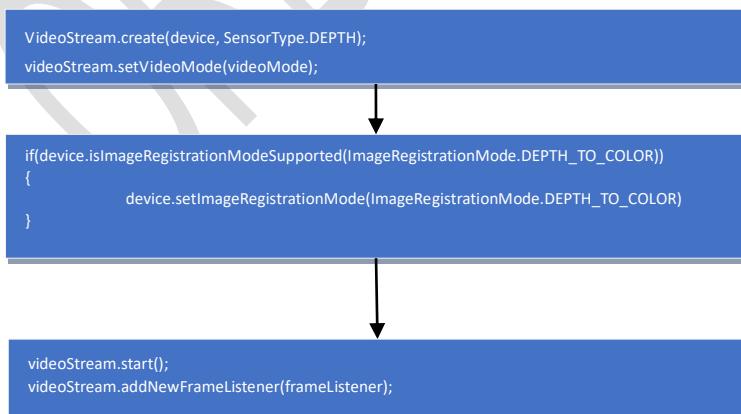


3. 规范 3，在 VideoStream 没有销毁的情况下，只需要执行 VideoStream 的 start 和 stop 函数即可，最优选择方案：



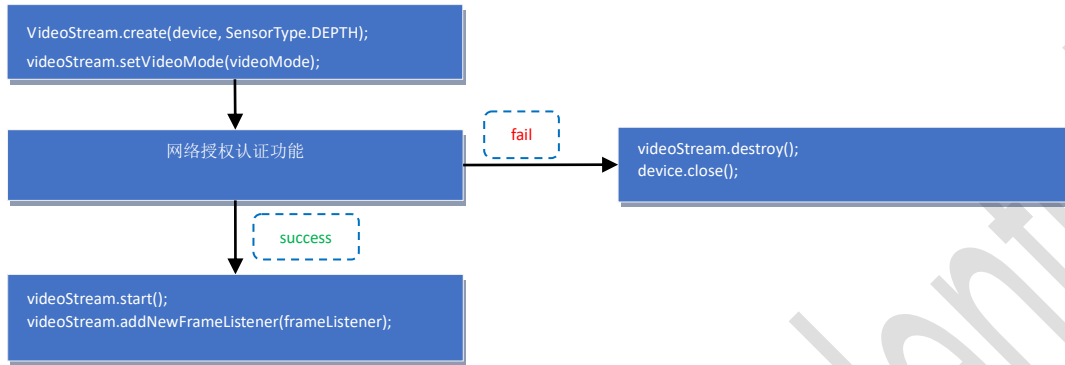
13.3 硬件 D2C 调用规范

设置硬件 D2C 必须在 VideoStream 创建之后调用，否则会出现调用异常错误现象。



13.4 网络授权规范

网络授权认证功能仅个别客户参考，其接口的调用过程中必须遵循流程规范，否则即使授权认证成功通过也无法获取到流数据。



14 SDK 释放

在 Android 应用程序中退出程序时候需要释放 SDK 所有资源，例如在 Activity 的 onDestory ()生命周期函数中去调用。最后退出整个 APP 时请调用步骤 12 对应的接口。

释放方式 1，逐个资源释放：

```

@Override
protected void onDestroy() {
    if(videoStream!=null){
        videoStream.stop();
        videoStream.removeNewFrameListener(newFrameListener);
    }

    if(videoStream!=null){
        videoStream.destroy();
        videoStream=null;
    }

    if(device!=null){
        device.close();
        device=null;
    }

    if(openNIHelper!=null){
        openNIHelper.shutdown();
    }

    OpenNI.shutdown();

    super.onDestroy();
}
  
```

释放方式 2：

删除数据流回调监听器，关闭 Device(关闭 Device 中会停止和释放 VideoStream)，释放所有资源：

```
@Override
protected void onDestroy() {
    if(videoStream!=null){
        videoStream.removeNewFrameListener(newFrameListener);
    }

    if(device!=null){
        device.close();
        device=null;
    }

    if(openNIHelper!=null){
        openNIHelper.shutdown();
    }

    OpenNI.shutdown();
    super.onDestroy();
}
```

15 纯净 IR 图获取

纯净 IR 如输出不是所有设备都支持，仅部分设备。例如 Gemini 等带有泛光灯的设备支持，获取纯净 IR 首先需要关闭激光，关闭 LDP，然后打开泛光灯，获取纯净 IR，其他流程与普通获取散斑 IR 一致，关闭激光与 LDP 接口都在扩展 API 中；

- [关闭激光接口](#)
- [关闭 LDP 接口](#)
- [打开泛光灯接口](#)

16 扩展 API 接口

16.1 Device 相关基础接口

16.1.1 获取设备信息

接口类	API	参数	描述
Device	getDeviceInfo()	无	返回 DeviceInfo 信息

16.1.2 设置帧同步

接口类	API	参数	描述
Device	setDepthColorSyncEnabled(boolean isEnabled)	true 打开，false 关闭	Color 通过 Openni2.0-sdk 设备支持，如 P1，其它设备不支持,P3X 不支持。
调用后 orbbec.ini 配置文件配置无效。			

16.1.3 获取帧同步状态

接口类	API	参数	描述
Device	getDepthColorSyncEnabled()	无	返回帧同步开关状态

16.1.4 设置激光状态

接口类	API	参数	描述
Device	setLaserEnable(boolean enable)	true 打开, false 关闭	打开或者关闭激光 须谨慎调用该接口, 无特殊需求 可不调用。

16.1.5 获取激光使能状态

接口类	API	参数	描述
Device	getLaserEnable()	无	返回 true 打开, false 关闭激光 须谨慎调用该接口, 无特殊需求可不调用。

16.1.6 设置 LDP 状态

接口类	API	参数	描述
Device	setLdpEnable(int ldpEnable)	true 打开, false 关闭	打开或者关闭 LDP 须谨慎调用该接口, 无特殊需求 可不调用。

16.1.7 设置 IR 泛光灯状态

接口类	API	参数	描述
Device	setIrFloodEnable(int enable)	true 打开, false 关闭	打开或者关闭泛光灯 须谨慎调用该接口, 无特殊需求 可不调用。

16.1.8 获取 IR 泛光灯状态

接口类	API	参数	描述
Device	getIrFloodEnable()	无	true 打开, false 关闭 须谨慎调用该接口, 无特殊需求 可不调用。

16.1.9 设置 IR 增益

接口类	API	参数	描述
Device	setGain(int gain)	gain 增益值	设置设备增益值大小

16.1.10 获取 IR 增益

接口类	API	参数	描述
Device	getGain()	无	返回设备增益大小

16.1.11 获取 USB 类型

接口类	API	参数	描述
Device	getUSBSpeed()	无	返回设备 USB 类型，具体参照 DeviceUSBSpeed

16.1.12 DeviceUSBSpeed

接口类	描述
DeviceUSBSpeed	Device USB 类型枚举类。 USB_LOW_SPEED = 0; USB_FULL_SPEED = 1; USB_HIGH_SPEED = 2; //USB2.0 USB_SUPER_SPEED = 3; //USB3.0

16.1.13 获取相机保存标定参数

接口类	API	参数	描述
Device	getOBCameraParams()	无	返回 OBCameraParams 相机标定参数

16.1.14 获取固件版本号

接口类	API	参数	描述
Device	getFirmwareVersion ()	无	返回设备固件版本号

16.1.15 获取设备 SN 号

接口类	API	参数	描述
Device	getSerialNumber()	无	返回设备 SN 号

16.1.16 硬件 D2C 支持判断

接口类	API	参数	描述
Device	isImageRegistrationModeSupported(ImageRegistrationMode mode)	ImageRegistrationMode	判断是否支持硬件 D2C，一般传参：DEPTH_TO_COLOR

16.1.17 设置硬件 D2C 模式

接口类	API	参数	描述
Device	setImageRegistrationMode(ImageRegistrationMode mode)	ImageRegistrationMode	设置硬件 D2C 模式，OFF 关闭，DEPTH_TO_COLOR 打开

16.1.18 获取硬件 D2C 模式

接口类	API	参数	描述
Device	getImageRegistrationMode()	无	返回硬件 D2C 模式：ImageRegistrationMode，OFF 关闭，DEPTH_TO_COLOR 打开

16.1.19 设置 Depth 和 IR 交替输出模式

接口类	API	参数	描述
Device	setDepthIrAlternateMode(int nMode)	int nMode (DepthIrMode)	设置 Depth 和 IR 交替输出模式，仅仅部分设备支持，如海燕 Pro、海燕 Plus、P3X，其它 USB 模组不支持。

参数	参数描述
int nMode	DepthIrMode MODE_DEPTH_NIR(0): Depth 和纯净 IR 模式，即：Depth 和纯净 IR 交错输出；（P3X 不支持）； MODE_SPECKLE_IR(1): Depth 和散斑 IR 模式，即：Depth 和散斑 IR 模式同时或单独输出； MODE_NIR(2): 纯净 IR 模式，即：只输出纯净 IR；

16.1.20 获取 Depth 和 IR 交替输出模式

接口类	API	参数	描述
Device	getDepthIrAlternateMode()	无	获取 Depth 和 IR 交替输出模式，仅仅部分设备支持，如海燕 Pro、海燕 Plus、P3X，其它 USB 模组不支持。 返回值： 0: Depth 和 IR 交错输出； 1: 散斑 IR 模式； 2: 纯净 IR 模式；

16.2 VideoStream 相关基础接口

16.2.1 设置镜像

接口类	API	参数	描述
VideoStream	setMirroringEnabled(boolean isEnabled)	boolean	设置 VideoStream 镜像状态，true 打开，false 关闭。
调用后 orbbec.ini 配置文件配置无效。			

16.2.2 获取镜像状态

接口类	API	参数	描述
VideoStream	getMirroringEnabled()	无	返回 VideoStream 镜像状态，true 打开，false 关闭。

16.2.3 设置 VideoMode

接口类	API	参数	描述
VideoStream	setVideoMode(VideoMode videoMode)	VideoMode	VideoStream 创建后设置。 设置分辨率、PixelFormat、帧率(FPS)。 分辨率：根据不同设备决定； PixelFormat： Depth(DEPTH_1_MM),IR(GRAY16),Color(RGB888)。 FPS:一般设置 30FPS，暂时不支持动态设置。
调用后 orbbec.ini 配置文件配置无效。			

参数	参数描述
----	------



PixelFormat	<pre>//Depth DEPTH_1_MM(100), DEPTH_100_UM(101), SHIFT_9_2(102), SHIFT_9_3(103), // Color RGB888(200), YUV422(201), GRAY8(202), GRAY16(203), JPEG(204), YUYV(205);</pre>
-------------	---

16.2.4 获取 VideoMode

接口类	API	参数	描述
VideoStream	getVideoMode()	无	获取当前 VideoStream 输出的 VideoMode 模式

16.2.5 设置 Depth 最大值

接口类	API	参数	描述
VideoStream	setStreamMaxDepth(int maxDepth)	int (单位:mm)	设置 Depth 最大值的目的是过滤掉 Depth 不再这个范围内的点的 Depth，需要在 VideoStream 的 create 函数和 start 函数之间设置，设置的参考代码和设置 Depth 的分辨率类似。

16.2.6 设置 Depth 最小值

接口类	API	参数	描述
VideoStream	setStreamMinDepth(int minDepth)	int (单位:mm)	设置 Depth 最小值的目的是过滤掉 Depth 不再这个范围内的点的 Depth，需要在 VideoStream 的 create 函数和 start 函数之间设置，设置的参考代码和设置 Depth 的分辨率类似。

16.2.7 设置数据输入格式

接口类	API	参数	描述
VideoStream	setStreamInputFormat(int inputFormat)	int	设置 VideoStream 的输入数据格式（固件到 SDK），在 VideoStream 的 create 函数和 start 函数之间设置。

参数	参数描述
inputFormat	<pre>//Depth DEPTH_FORMAT_16_BIT(0), DEPTH_FORMAT_10_BIT(2),</pre>



	<pre> DEPTH_FORMAT_11_BIT(3), DEPTH_FORMAT_12_BIT(4), // IR IR_FORMAT_16_BIT(0), IR_FORMAT_10_BIT(2), // Color COLOR_FORMAT_YUV422(5), COLOR_FORMAT_MJPEG(8); </pre>
--	--

16.2.8 设置 HoleFilter

接口类	API	参数	描述
VideoStream	setStreamHoleFilter(int holeFilter)	int	填洞滤波，0：表示关闭滤波功能，开启时取值 1，2，3，4 (HoleFilter 代表窗口滤波器的大小，1 是 3*3，2 是 5*5，3 是 7*7，4 是 9*9)，在 VideoStream 的 create 函数和 start 函数之间设置。

16.2.9 设置 SoftFilter

接口类	API	参数	描述
VideoStream	setSoftFilter(SoftFilterType softFilter)	SoftFilterType	软件滤波，调用该接口需谨慎，会消耗 Android 平台资源，如影响深度数据的接收。

参数	参数描述
SoftFilterType	CLOSE(0): 关闭软件滤波，默认不调用该接口是关闭状态； OPEN(2): 打开软件滤波。

17 配置文件介绍

17.1 Openni.ini 配置文件

Openni.ini 配置文件
<pre> [Log] ; 0 - Verbose; 1 - Info; 2 - Warning; 3 - Error. Default - None ;Verbosity=0 //输出 Log 的等级 </pre>



```
;LogToConsole=1 //输出 Log 到控制台
;LogToFile=1 //输出 Log 到文件
;LogToAndroidLog=1 //在 Android 平台输出 log
```

17.2 Orbbec.ini 配置文件

Openni.ini 配置文件

```
;----- Sensor Default Configuration -----
[Device]
; Mirroring. 0 - Off , 1 - On (default)
;设置 Device 的 mirror 属性, depth 和 ir、color 默认 mirror。
;Mirror=1

; FrameSync. 0 - Off (default), 1 - On
;设置 depth 和 color 帧同步, 只有 color 是 openni 协议传输的设备支持帧同步
;FrameSync=1

; Stream Data Timestamps. 0 - milliseconds, 1 - microseconds (default)
;设置时间戳的单位, 这个地方不用修改, 用默认值。(0:表示 ms, 1: 表示微秒)
;HighResTimestamps=1

; Stream Data Timestamps Source. 0 - Firmware (default), 1 - Host
;设置时间戳源, 不用修改, 用默认值, 用固件的时间戳 (0:固件时间戳,1:主机的时间戳)
;HostTimestamps=0

; USB interface to be used. 0 - FW Default, 1 - ISO endpoints (default on Windows), 2 - BULK endpoints (default on Linux/Mac/Android machines), 3 - ISO endpoints for low-bandwidth depth
;usb 传输类型, 值为 2, 不能修改
UsbInterface=2

[Depth]
; Output format. 100 - 1mm depth values (default), 102 - u9.2 Shift values.
;设置 depth 的输出格式 100: 表示 1mm 精度; 101: 100um 精度; 102: 表示输出原始视差;
;OutputFormat=100

; Is stream mirrored. 0 - Off, 1 - On
;覆盖 Device 的 Mirror 值, depth 镜像, 0: 表示关闭镜像; 1: 表示打开镜像;
;Mirror=1
```



; 0 - QVGA, 1 - VGA, 4 - QQVGA. Default: Arm - 4, other platforms - 0, -14 1280x720, 15 1280x960,-16 1280x800, -17 640x400 -20 320x200,-21 480x640, -25 960x1280, -26 800x1280,-27 400x640

;设置 Depth 的分辨率;

Resolution=1

; Frames per second (default is 30)

; 设置 depth 的帧率(目前不支持动态设置分辨率)

;FPS=30

; Min depth cutoff. 0-10000 mm (default is 0)

; 过滤的最小深度值, 小于该深度值, 过滤掉, 置为 0。

; 举例: (如: MinDepthValue =300, 那么小于 30 厘米的 depth 就会过滤掉)

;MinDepthValue=0

; Max depth cutoff. 0-10000 mm (default is 10000)

; 过滤最大的深度值, 大于该值的 depth 会过滤掉, 置为 0 。

;MaxDepthValue=10000

; Input format. 0 - Uncompressed 16-bit, 1 - PS Compression,2 - Packed 10-bit, 3 - Packed 11-bit, 4 - Packed 12-bit. Default: Arm - 4, other platforms - 3

;深度打包格式, 现在我们用到的有 (10bit, 11bit, 12bit) 其中 3: 表示 11bit; 4: 表示 12bit; 2: 表示 10bit (D2 和 Atlas,Mipi 公板 设置成 2)

;InputFormat=3

; software filter 0-off ,1--on (default)

;软件滤波开关, 2.3.0.55 SDK 及后续版本支持

;SoftFilter=0

; Depth Rotate 0-off(default) ,1--on (only atlas device support)

;Depth 旋转功能, 在 2.3.0.61 版本支持, 只有 Atlas 设备有这个功能

;DepthRotate=0

; Hole Filler. 0 - Off, 1 - On (default)

; 填洞滤波, 0: 表示关闭滤波功能, 开启时取值 1, 2, 3, 4 : (HoleFilter 代表窗口滤波器的大小, 1 是 3*3, 2 是 5*5, 3 是 7*7, 4 是 9*9)

;HoleFilter=0

[Image]

;整个[Image]字段只有对走 Openni 协议 rgb 有效, 对 UVC 协议的无效;



; Output format. 200 - RGB888 (default), 201 - YUV422, 202 - Gray8 (2.0 MP only), 205 - YUYV
;设置 color 图像的输出格式, 只有对 openni 输出的有效, 目前支持 RGB888(200)和 YUV422(201);
;OutputFormat=201

; Is stream mirrored. 0 - Off, 1 - On
;覆盖 Device Mirror 值, color 图像的镜像方式; 0: 非镜像; 1: 镜像
;Mirror=1

; 0 - QVGA (default), 1 - VGA, 2 - SXGA (1.3MP), 3 - UXGA (2.0MP), 14 - 720p, 15 - 1280x960
;设置 color 图像的分辨率
Resolution=1

; Frames per second (default is 30)
;设置 color 的帧率, 目前我们大部分产品都是恒定帧率, 不支持动态设置分辨率
;FPS=30

; Input format. 0 - Compressed 8-bit BAYER (1.3MP or 2.0MP only), 1 - Compressed YUV422 (default in BULK), 2 - Jpeg, 5 - Uncompressed YUV422 (default in ISO), 6 - Uncompressed 8-bit BAYER (1.3MP or 2.0MP only), 7 - Uncompressed YUYV, 8 - MJPEG
;设置 color 图像的打包格式, 目前设置成 5,不要修改,P3X 输出 MJPEG 格式
InputFormat=5

[IR]
; Output format. 200 - RGB888, 203 - Grayscale 16-bit (default)
;设置 IR 的输出格式
;OutputFormat=203
; Is stream mirrored. 0 - Off, 1 - On
;覆盖 Device 字段的 Mirror 值, 设置 IR 镜像 0: 表示非镜像; 1: 表示镜像
;Mirror=1
; 0 - QVGA (default), 1 - VGA, 2 - SXGA(1.3MP), 4 - QQVGA. -14 1280x720, 15 1280x960,-16 1280x800, -17 640x400 -20 320x200,-21 480x640, -25 960x1280, -26 800x1280,-27 400x640
;设置 IR 分辨率, 分辨率列表和 depth 一样
Resolution=1
; Frames per second (default is 30)
;设置帧率, 暂时不支持动态设置; ;FPS=30



18 Libuvc 应用

Android 平台 Orbbec 模组 RGB 如果是 UVC 设备需要使用第三方 libuvc 应用打开，libuvc 使用基于 GitHub 开源框架 UVCCamera: <https://github.com/saki4510t/UVCCamera>。

详细使用请参考官方源码，下面只列出如何在 libuvc 中应用回调方式将原数据返回应用层供客户使用。

18.1 数据监听器注册

设置 RGB 数据回调监听器

```
camera.setFrameCallback(mIFrameCallback, UVCCamera.FRAME_FORMAT_MJPEG);
```

18.2 数据回调

RGB 数据回调监听器

```
private final IFrameCallback mIFrameCallback = new IFrameCallback() {  
    //该方法没有在 UI 线程中  
    @Override  
    public void onFrame(final ByteBuffer frame) {  
    }  
};
```

19 Demo 参考

OpenNI2.0-SDK 版本发布附带 Demo 源码供客户参看使用，此 Demo 分别以主动和回调各三种方式去调用 SDK 对应的 API 接口，请参考并正确使用 API。

如下路径：



20 结束

《Orbbec OpenNI2.0-SDK Android 接口使用规范》会根据 Orbbec-sdk 的变更不断的修改和优化。

ORBBEC Confidential